Programmering med mikrokontroller

Aut2602 Innlevering 4

Jens Christian Valen Leynse

Oktober 2024

1 Kode segmenter

main.c

```
#define F_CPU 16000000UL

#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
#include <avr/cpufunc.h>
#include <stdio.h>
#include "usart.h"
#include "adc.h"


#define MAX_FREQ_DIM 40000
uint16_t ADC_MIN = 4096;
uint16_t ADC_MAX = 0;


void clock_frequenz()
{
        ccp_write_io (( void *) & ( CLKCTRL . OSCHFCTRLA ) ,
            CLKCTRL_FRQSEL_16M_gc | 0 < CLKCTRL_RUNSTDBY_bp ) ;
}


void PWM_init()
{
        TCA0.SINGLE.PER = MAX_FREQ_DIM;

        PORTC.PORTCTRL = PORT_INVEN_bm;

        TCA0.SINGLE.CTRLA = TCA_SINGLE_ENABLE_bm | TCA_SINGLE_CLKSEL_DIV4_gc;

        // Task A
        TCA0.SINGLE.CTRLB = TCA_SINGLE_WGMODE_SINGLESLOPE_gc |
            TCA_SINGLE_CMP0EN_bm | TCA_SINGLE_CMP1EN_bm |
            TCA_SINGLE_CMP2EN_bm;
}
```

```
void Servo_init()
{
        TCA0.SINGLE.PER = MAX_FREQ_DIM;

        TCA0.SINGLE.CTRLA = TCA_SINGLE_ENABLE_bm | TCA_SINGLE_CLKSEL_DIV8_gc;

        TCA0.SINGLE.CTRLB = TCA_SINGLE_WGMODE_SINGLESLOPE_gc |
            TCA_SINGLE_CMP0EN_bm | TCA_SINGLE_CMP1EN_bm |
            TCA_SINGLE_CMP2EN_bm;
}

// Task B
void switch_PWM_output(uint8_t swap)
{
        PORTMUX.TCAROUTEA = swap;

        switch (swap){
        case PORTMUX_TCA0_PORTC_gc:
                PORTC.PORTCTRL = PORT_INVEN_bm;
                PORTC.DIR |= PIN0_bm | PIN1_bm | PIN2_bm;
                printf("switching␣to␣C\n");
                break;
        case PORTMUX_TCA0_PORTE_gc:
                PORTE.PINCONFIG = PORT_INVEN_bm;
                PORTE.PINCTRLSET = PIN0_bm;
                PORTE.DIR |= PIN0_bm | PIN1_bm | PIN2_bm;
                printf("switching␣to␣E\n");
                break;
        }
}

void set_pulse_width(uint16_t pulseWidth, uint16_t PWM_MAX, uint16_t PWM_MIN)
{
        // Cap pulse width to defined limits
        if (pulseWidth < PWM_MIN) pulseWidth = PWM_MIN;
        if (pulseWidth > PWM_MAX) pulseWidth = PWM_MAX;

        TCA0.SINGLE.CMP0 = pulseWidth;  // Set the PWM duty cycle
}

uint16_t map_adc_to_pwm(uint16_t adc_value, uint16_t PWM_MAX, uint16_t
    PWM_MIN)
{
        if (adc_value < ADC_MIN) ADC_MIN = adc_value;
        if (adc_value > ADC_MAX) ADC_MAX = adc_value;

        float normalized = (float)(adc_value - ADC_MIN) / (ADC_MAX - ADC_MIN)
            ;
    return (uint32_t)(normalized * (PWM_MAX - PWM_MIN) + PWM_MIN);
}

int main(void)
{
        char Task;
        uint16_t  freq = MAX_FREQ_DIM;

        clock_frequenz();
```

```
        usart_usb_init();
        ADC_Init();
        sei();

        uint16_t pwm_min = MAX_FREQ_DIM / 20;
        uint16_t pwm_max = MAX_FREQ_DIM / 10;

        // Switch what assignment task to test here (either C, D, E and F)
        Task = 'E';

        // switches functions between LEDs and transistors
        if (Task == 'C') {      // LEDs
                PORTC.DIR = PIN3_bm;
                PWM_init();
                switch_PWM_output(PORTMUX_TCA0_PORTC_gc);
        } else {                        // Transistors and servo
                Servo_init();
                switch_PWM_output(PORTMUX_TCA0_PORTE_gc);
        }

        while (1) {
                switch (Task) {
                case 'C':
                        if (freq >= MAX_FREQ_DIM) {
                                freq = 0;
                                } else {
                                freq += 500;
                        }
                        _delay_ms(50);
                        TCA0.SINGLE.CMP0 = TCA0.SINGLE.CMP1 = TCA0.SINGLE.
                            CMP2 = freq;
                        break;

                case 'D':
                        // Finn andre PER og CMP slik at CMP blir s   stort
                            som mulig
                        TCA0.SINGLE.CMP0 = pwm_max;
                        _delay_ms(1000);
                        TCA0.SINGLE.CMP0 = pwm_min;
                        _delay_ms(1000);

                        /*
                        // St rst mulig CMP ->

                        F_CPU 16M, F_PWM = 50 Hz, omr de mellom 1 og 2 ms
                        F_CPU / (DIV * F_PWM) -> DIV 8 -> PER = 40000

                        Oppl sning p   40000 totalt.
                        50 Hz tilsvarer 20ms

                        20ms -> 40000   1ms -> 40000/20 = 2000  2ms -> 4000
                        CMP mellom 2000 og 4000. Oppl sning blir 2000.

                        DIV 64 -> PER = 5000 ->  1ms = 250      2ms = 500
                        DIV 32 -> PER = 10000 -> 1ms = 500      2ms = 1000
                        DIV 16 -> PER = 20000 -> 1ms = 1000     2ms = 2000
                        */
```

```
                break;

        case 'E':
                ADC_Start();

                uint16_t adc_value = adc_get_result();

                uint16_t pulseWidth = map_adc_to_pwm(adc_value,
                    pwm_max, pwm_min);

                printf("ADC value: %u, ADC_MIN: %u, ADC_MAX: %u,
                    PWM_MIN: %u, PWM_MAX: %u\n",
                adc_value, ADC_MIN, ADC_MAX, pwm_min, pwm_max);

                set_pulse_width(pulseWidth, pwm_max, pwm_min);

                _delay_ms(1000);

                /*
                ADC-oppl sningen avgj r hvor mange forskjellige
                    analoge
                inngangsverdier som kan leses. H yere oppl sning
                    gir mer
                presise m linger fra potensiometeret, som deretter
                    kan
                mappe til PWM-verdier mer n yaktig.
                */
                break;

        case 'F':
                // write <"puls width"> in the terminal (ex. <1500>)
                if (usb_message_ready) {
                        int value = atoi(usb_message);

                        // Check if the input value is within the
                            valid range (1ms to 2ms)
                        if (value < 1000 || value > 2000) {
                                // Handle invalid input (e.g., print
                                    an error message)
                                printf("Error: Pulse width must be
                                    between 1000 s and 2000 s .
                                    Received: %d\n", value);
                        } else {
                                uint32_t pulse = (pwm_min * (uint32_t
                                    )value) / 1000; // Convert to ms

                                uint16_t pulseWidth = (uint16_t)pulse
                                    ;

                                set_pulse_width(pulseWidth, pwm_max,
                                    pwm_min);
                        }

                        usb_message_ready = 0;
                }
                break;
        }
```

```
        }
}
```

adc.c

```
#include "adc.h"
#include <avr/io.h>

#define SCALING_FACTOR 4096

void ADC_Init(void) {
        VREF.ADC0REF = VREF_REFSEL_2V048_gc | VREF_ALWAYSON_bm;

        // Set ADC clock, sample length, and initialization delay
        ADC0.CTRLA = ADC_ENABLE_bm | ADC_FREERUN_bm | ADC_RESSEL_12BIT_gc; //
             Enable ADC, freerunning makes startconversion only necessary in
            init
        ADC0.CTRLC = ADC_PRESC_DIV16_gc; // Set prescaler for 250kHz
        ADC0.MUXPOS = ADC_MUXPOS_AIN4_gc;
}

// Start the conversion
void ADC_Start(void) {
        ADC0.COMMAND = ADC_STCONV_bm;
}

uint16_t adc_get_result(){
        while(!ADC0.INTFLAGS & ADC_RESRDY_bm)
                ;
        return ADC0.RES;
}
```

adc.h

```
#ifndef ADC_H_
#define ADC_H_

#include <avr/io.h>
#include <stdbool.h>

void ADC_Init(void);
void ADC_Start(void);
uint16_t ADC_GetResult(void);
#endif /* ADC_H_ */
```

usart.c

```
/*
 * usart.c
 *
 * Created: 24.08.2022 14:14:12
 *  Author: pol022
 */
```

```
#include <avr/io.h>
#include "usart.h"
#include <avr/interrupt.h>


void usart_usb_init(){
        /* Necessary configuration*/
        USART3.BAUD = 6667; // Baudrate 9600 // F_CPU = 16M
        USART3.CTRLB |= USART_RXEN_bm | USART_TXEN_bm |
            USART_RXMODE_NORMAL_gc; //enable RX TX
        PORTB.DIR |= (1<<TX_PIN_bp);
        /* interrupt on RX*/
        USART3.CTRLA |= USART_RXCIE_bm;

        /* interrupt on TX */
        //USART3.CTRLA |= USART_TXCIE_bm;

        /* printf */
        stdout = &new_std_out; // address to struct
}


void usart_usb_transmit(char c){
        while(!(USART3.STATUS & USART_DREIF_bm)){
                ;
        }
        USART3.TXDATAL = c;
}


/* transmitting*/
void usart_usb_transmit_char_array(uint8_t addEndLine,char string[], uint8_t
    length){
        for(uint8_t i = 0; i<length; i++){
                usart_usb_transmit(string[i]);
        }
        if(addEndLine)
                usart_usb_transmit('\n');
}

/* rx-polling*/
char usart_usb_receive(){
        while(!(USART3.STATUS & USART_RXCIF_bm))// not unread data in
            receiver
                ;
        return USART3.RXDATAL;
}


/* rx-interrupt with some handling.*/
ISR(USART3_RXC_vect){
        char tmp = USART3.RXDATAL;
        switch (tmp)
        {
                case '<':
                        marker_pos = 0;
```

```
                        break;
                case '>':
                        usb_message_ready = 1;
                        usb_message[marker_pos] = 0;
                        break;
                default:
                        usb_message[marker_pos] = tmp;
                        marker_pos++;
        }
}

static uint8_t usart_usb_transmit_printf(char c, FILE *stream){
        usart_usb_transmit(c);
        return 0;
}


/* Generic usart-files */
void usart_init(USART_t *usart_nr, PORT_t *usart_port, uint8_t tx_bp, uint8_t
     rx_bp){
        /* Necessary configuration*/
        //PORTMUX.USARTROUTEA |= PORTMUX_USART2_ALT1_gc;// non generic
        usart_nr->BAUD = 1667; // Baudrate 9600
        usart_nr->CTRLB |= USART_RXEN_bm | USART_TXEN_bm |
            USART_RXMODE_NORMAL_gc; //enable RX TX
        usart_port->DIR |= (1<<tx_bp);
        usart_port->PINCONFIG |= PORT_PULLUPEN_bm;
        usart_port->PINCTRLSET |= (1<<rx_bp);
        // RX-handling in separate function.
        //usart_port->CTRLA |= USART_RXCIE_bm; // interrupt on RX
}

void usart_transmit_char(USART_t *usart_nr,char c){
        while(!(usart_nr->STATUS & USART_DREIF_bm)){
                ;
        }
        usart_nr->TXDATAL = c;
}
void usart_transmit_string(USART_t *usart_nr, char *string,
    usart_char_transmitter transmit_char){
        for(uint8_t i = 0; string[i] != '\0'; i++)
        transmit_char(usart_nr, string[i]);
}

// example with USART2
ISR(USART2_RXC_vect){
        char tmp = USART2.RXDATAL;
        switch (tmp)
        {
                case '{':
                        u2_marker_pos = 0;
                        break;
                case '}':
                u2_message_ready = 1;
                u2_message[marker_pos] = 0;
                break;
                default:
```

```
                u2_message[marker_pos] = tmp;
                u2_marker_pos++;
        }
}
```

usart.h

```
/*
 * usart.h
 *
 * Created: 24.08.2022 14:14:25
 *  Author: pol022
 */


#ifndef USART_H_
#define USART_H_

#include <avr/io.h>
#define TX_PIN_bp 0
#define RX_PIN_bp 1
#include <stdio.h>

char usb_message[256];
volatile uint8_t marker_pos;
volatile uint8_t usb_message_ready;


volatile uint8_t u2_marker_pos;
volatile uint8_t u2_message_ready;
char u2_message[256];



void usart_usb_init();

void usart_usb_transmit(char c);
char usart_usb_receive();

/* printing an array of chars*/
void usart_usb_transmit_char_array(uint8_t addEndLine,char string[], uint8_t
    length);



/* preparing for printf */
static uint8_t usart_usb_transmit_printf(char c, FILE *stream);
static FILE new_std_out = FDEV_SETUP_STREAM(usart_usb_transmit_printf, NULL,
    _FDEV_SETUP_WRITE);


/* generic usart files */
void usart_init(USART_t *usart_nr, PORT_t *usart_port, uint8_t tx_bp, uint8_t
     rx_bp);
void usart_transmit_char(USART_t *usart_nr,char c);
typedef void (*usart_char_transmitter)(USART_t *usart_nr,char c);
```

```
void usart_transmit_string(USART_t *usart_nr, char *string,
    usart_char_transmitter transmit_char);

#endif /* USART_H_ */
```